

# AI Software Engineer Case Study: RAG / Vector Search

## 1. The task

Build a small full-stack application in TypeScript, organized as a monorepo, that indexes a corpus of documents into a vector store and lets users search it semantically. On top of retrieval, generate grounded answers (RAG) with citations back to the source documents.

The application has two surfaces:

- **Chat Page:** an end-user experience for asking a question in natural language, getting relevant passages back, and receiving a grounded answer with citations to the documents it came from.
- **Dashboard:** an authenticated view for managing the corpus and observing the system: document and ingestion status, index health, and basic search analytics.

Search must also be exposed as an MCP (Model Context Protocol) server, so the same retrieval capability can be called as a tool by an external MCP client.

Access is gated by authentication and authorization: users sign in, and what they can see and do depends on their role (for example, a regular user can search, while an admin can manage documents and view the dashboard).

You may extend or reshape the scope. If you make a meaningful design decision (chunking strategy, embedding model, vector store, retrieval approach), explain why in a sentence or two.

A sample dataset (the corpus to index) is provided with this case. Design your ingestion so that pointing it at the provided corpus is straightforward.

## 2. Must-haves

Keep it focused. A smaller system that works end-to-end beats a large one that half-works.

- **Monorepo.** Frontend, backend, and any shared code (types, schemas) live in one repository with a clear workspace structure and shared TypeScript types across the boundary.
- **Ingestion pipeline.** Take the source documents, chunk them, generate embeddings, and store them in a vector store. Ingestion should be repeatable and observable: it is clear what was indexed, when, and whether it succeeded.
- **Semantic search + RAG.** Retrieve relevant chunks for a query via vector similarity, then produce a grounded answer that cites the source documents. When the corpus does not contain the answer, say so instead of making something up. Retrieval quality matters more than answer eloquence.
- **Chat Page.** An end-user page to ask a question and see results and the grounded answer with citations.
- **Dashboard.** An authenticated view to manage the corpus (see indexed documents, trigger or observe ingestion) and view basic system and search stats.

- **MCP server for search.** Expose search as an MCP tool that an MCP client can call and get results back. Document how to connect to it.
- **Authentication and authorization.** Users sign in; access to pages, actions, and APIs is enforced by role. Protect the dashboard and management actions from regular users.
- **AI usage log.** What you had AI do, what you wrote yourself, where AI got it wrong, and how you caught it. A short markdown file is enough.
- **README.** Enough for us to run the whole thing on a fresh machine (see Required Documentation below).

### 3. Technical requirements

- TypeScript is required for the entire project.
- Use Tailwind CSS for styling.
- Design responsive layouts that work on mobile phones, tablets, and desktop computers.

Development standards: use clean, readable code with consistent formatting. Implement proper error handling throughout the application. Create reusable components and follow your chosen framework's best practices. Use TypeScript interfaces for data structures and API responses. Write meaningful commit messages and maintain a clean Git history.

### 4. Required Documentation

Keep the README simple and to the point. It should include:

- **Project Description:** brief overview of what the application does and its main features.
- **Technology Stack:** all technologies, frameworks, and libraries used in the project.
- **Installation Instructions:** step-by-step guide to set up the project locally.
- **Running the Application:** clear instructions on how to start both frontend and backend servers.
- **Demo Credentials:** test user accounts for both regular user and admin roles.
- **API Documentation:** main API endpoints with brief descriptions and example requests.
- **Deployment Guide:** if you did the deployment bonus, how it is deployed; otherwise a few sentences on how you would deploy it.
- **Features List:** implemented features and any bonus features added.

Include example environment variable files and basic database seeding instructions to help with setup.

### 5. Bonus (optional)

None of these are required. Skipping them does not lower your score; doing them can raise it.

- **MCP authentication via OIDC.** Secure the MCP server with OpenID Connect so that only authenticated, authorized clients can call the search tool. This is considered a significant bonus.
- **Self-updating pipeline.** Ingestion that keeps the index current on its own: detecting new, changed, or removed documents and re-indexing incrementally without a full manual rebuild. This is considered a significant bonus.
- Live deployment: a working deployment with a public demo link.
- Improvements to retrieval quality, such as hybrid (keyword + vector) search, reranking, or query rewriting.
- A small evaluation that measures retrieval or answer quality across a set of queries and reports the results.
- Streaming answers, result highlighting, or other search-experience polish.
- User management: an admin surface to invite, list, and manage users and their roles.

We want you to make this case your own. Extending the scope, tightening a requirement, or adding something that makes the system genuinely better is encouraged; tell us what you added and why.

## 6. Rules and practical notes

- The system should run locally on a fresh machine following your README. Any external services required (vector store, model provider) must be documented, and any keys supplied via environment variables with an example file.
- Using AI while you build (Claude, Copilot, Cursor, etc.) is expected and welcome. Record it in the AI usage log.
- A sample dataset is provided with the case. Design your ingestion so pointing it at the real corpus is straightforward.
- Any libraries, frameworks, and vector stores you like.
- Timebox: about two days of work. Scope accordingly; a focused system beats an unfinished ambitious one.
- Submit your work as an invite to a private GitHub repository with clean commit history and meaningful commit messages.
- Please do not publish the case document, the dataset, or your solution publicly.
- Questions while working? Reply to the email you received this case from.

## 7. Deliverables

- Complete source code, delivered as an invite to a private GitHub repository.
- README.md: run instructions for a fresh machine, plus your design choices (see Required Documentation).
- AI usage log as a separate file (e.g. AI\_USAGE.md).
- Example environment variable files and seeding instructions.
- If you did the deployment bonus: the live demo link and how it is deployed.

## 8. What's in the package

- This document (task and requirements).
- The sample dataset (*sample\_dataset.zip*) to index, with a few example queries.

## 9. How we evaluate

Equally weighted:

- Retrieval and RAG design and quality
- Monorepo and system architecture
- Code quality
- Security (authentication and authorization)
- Clarity of communication (README, commit history, and AI usage log)

Responsive UI and error handling are expected throughout. Bonus items can raise a score; skipping them does not lower one. If you move forward, the interview includes a walkthrough of your code and AI usage log, so be ready to explain your decisions and change things live.